

Q Rotated Array By k

Brute force:-
 $T.C \rightarrow O(n \cdot k)$
 $S.C \rightarrow O(1)$
 [1, 2, 3, 4, 5, 6, 7], $k=3$
 After 3 rotation arr look like
 [5, 6, 7, 1, 2, 3, 4]

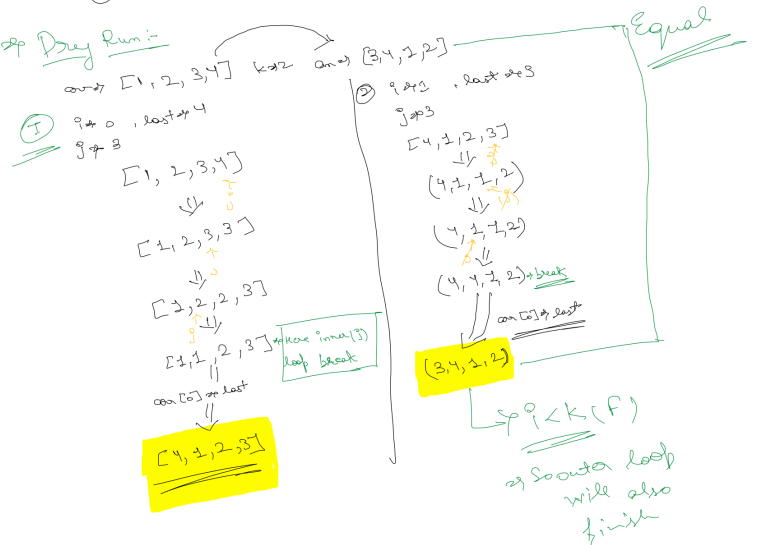
Problem

It works ok, but $T.C$ become high when k is large.

Pseudo Code

- 1) $K \rightarrow K \% n$ help to reduce unnecessary rotation
- 2) for $(i=0; i < K; i++)$
- 3) $\text{int last} = \text{arr}[n-1];$
- 4) for $(j=n-1; j >= 0; j--)$
- 5) $\text{arr}[j] \rightarrow \text{arr}[j-1];$
- 6) $\text{arr}[0] \rightarrow \text{last};$
- 7) 3
- 8) 3

Dry Run:-



Optimized way:-

These main 3 step $T.C \rightarrow O(n)$
 $S.C \rightarrow O(1)$

- 1) reverse whole arr
 - 2) reverse the first K part
 - 3) reverse the remaining all.
- $\rightarrow$ This is called reverse algorithm

Pseudo Code

- 1) $K \rightarrow K \% n$
- 2) reverse (nums, 0, n-1);
- 3) reverse (nums, 0, k-1);
- 4) reverse (nums, k, n-1);
- 5) static void reverse(int[] nums, int s, int e)
- 6) while (s < e)
- 7) swap (arr[s], arr[e]);
- 8) s++; e--;
- 9) 3
- 10) 3

Dry Run

